Arts Sciences et Technologies et Métiers
CER Aix-en-Provence

IA et Apprentissage Machine
Arnaud Polette - arnaud.polette@ensam.eu
Mathias Kleiner - mathias.kleiner@ensam.eu

Travaux pratiques - IA et Apprentissage Machine

Logiciel à paramétrer et librairie Python à Installer (sur les machines de l'école) :

1) Mettre à jour `scikit-learn`
- Dans "Effectuer une recherche" sous Windows écrire "prompt" et lancer "Anaconda Prompt".
- Dans la fenêtre qui s'ouvre écrire : `pip install --upgrade --user scikit-learn` et valider
- Attendre la confirmation "Successfully installed"

2) Paramétrer Geany :
- Ouvrir Geany et créer un fichier code Python, en s'assurant que l'extension du fichier soit ".py".
- Ouvrir la fenêtre de configuration dans le menu "Construire" → "Définir les commandes de construction".
- Remplacer la commande d'exécution (1ère colonne avant dernière ligne) par `py "%f"`
- Vérifier que Geany arrive à exécuter une commande Python

Dans cette série de TP, trois sujets sont à traiter :

Partie 1 : *Prise en main de SkLearn*, l'objectif de ce sujet est de se familiariser avec la librairie SkLearn.

Partie 2 : *Classification de matériaux*, dans cette partie l'objectif sera de construire un modèle de classification de matériaux permettant de prédire un type de verre à partir de sa composition.

Partie 3 : *Consommation d'énergie et données météo*, dans cette dernière partie l'objectif sera d'utiliser une base de données construite sur le campus d'Aix afin d'étudier les liens entre la consommation d'énergie du campus et différentes caractéristiques, notamment les conditions météo, et de tenter de prédire la consommation du campus à partir de ces valeurs.

Un livrable est à fournir pour les parties 2 et 3 (la partie 1 est un sujet de découverte sans livrable), chaque livrable doit contenir un code commenté ainsi d'un document (word ou autre) expliquant votre démarche et vos résultats pour chaque point du sujet.

Les fichier ressources contiennent deux types de fichiers, les fichiers de départ (.py) et les jeux de données nécessaires (.csv) :

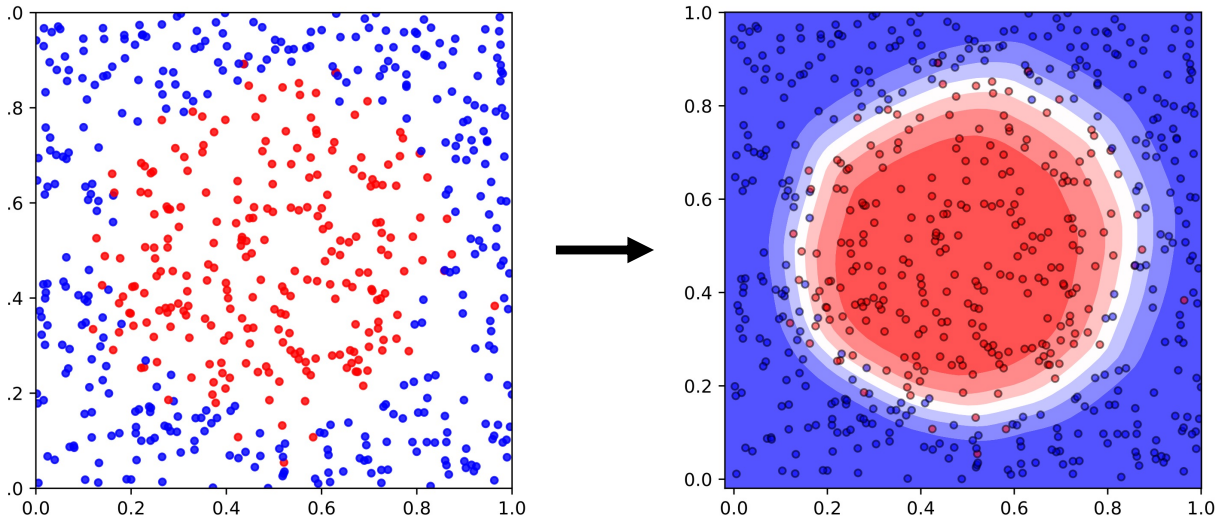
sujet1.py : fichier de départ du sujet 1, il utilise data.csv
data_1.csv : nuage de 1200 points 2D (x, y), avec une classe 0 ou 1.

sujet2.py : fichier de départ du sujet 2, il utilise glass_data.csv
glass_data.csv : caractéristiques de 214 pièces de verre et leur type.

sujet3.py : fichier de départ du sujet 3, il utilise conso_campus_aix.csv
conso_campus_aix.csv : données météos et consommation du campus d'Aix heure par heure sur un an.

Tous ces fichiers sont disponibles sur SAVOIR et sont à télécharger pour effectuer les différents sujets.

Sujet 1 - Prise en main de SkLearn, paramétrage, jeux de test, et score



Partie 1 : Prise en main de SkLearn.

L'archive contient un fichier `sujet1.py`, c'est un exemple simple qui charge le jeu de données et effectue un entrainement sur un réseau à un seul neurone, et calcule le score sur le jeu d'apprentissage et le jeu de test (à partir du fichier `data.csv`).

L'explication détaillée du code est disponible ci-dessous :

```
import numpy as np
from sklearn.neural_network import MLPClassifier
```

Importations les librairies nécessaires :
numpy : pour charger, stocker et traiter nos données
sklearn : pour construire et entrainer des réseaux de neurones

```
# chargement des données fichier csv de 3 colonnes par 1200 lignes
data = np.genfromtxt('data_1.csv', delimiter=',')
valeurs = data[:, 0:2]  # colonnes 1 et 2
classe = data[:, 2]  # colonne 3
```

data_1.csv contient 3 colonnes : x, y, et classe
Après chargement : **valeurs** = [[x1, y1], [x2, y2], [x3, y3], ...]
Après chargement : **classe** = [c1, c2, c3, ...]

```
# création du réseau
mlp = MLPClassifier(hidden_layer_sizes=(1,), solver='lbfgs', max_iter=10000)
```

hidden_layer_sizes=(1,) pour une seule couche à un neurone
solver='lbfgs' comme solver
max_iter= 10000 nombre d'itérations max haute

```
# apprentissage
mlp.fit(valeurs[0:10], classe[0:10])
```

Exécution de l'apprentissage sur les 10 premières données avec **fit()**

```
# score sur l'apprentissage
print("Score train :", mlp.score(valeurs[0:10], classe[0:10]))
# score sur un jeu de test
print("Score test :", mlp.score(valeurs[10:20], classe[10:20]))
```

Calcul du score sur le jeu d'apprentissage (10 premières valeurs)

Calcul du score sur le jeu de test (10 valeurs suivantes)

Les hyper-paramètres choisis dans ce code ne sont pas adaptés pour effectuer une bonne classification, dans cette première partie nous nous donnons comme objectif d'effectuer plusieurs modifications et rajouts dans ce programme afin de le rendre fonctionnel et exploitable.

Avant de commencer toute modification, vérifiez que le code s'exécute, et affiche bien deux scores (même si ils ne sont pas très hauts pour le moment).

Plusieurs choix sont à faire pour bien paramétrer notre apprentissage, et améliorer notre programme :

1. la forme et la taille de notre réseau de neurones : le paramètre **hidden_layer_sizes** permet de définir la taille des couches de notre réseau (voir doc MLPClassifier), une trop petite valeur peut mener à un sous-apprentissage, tandis qu'une trop grande à un sur-apprentissage (dans cet exemple des valeurs entre 1 et 100 sont intéressantes à essayer).

2. le solver : c'est ce qui détermine la stratégie d'optimisation des poids lors de l'apprentissage, ils peuvent être plus ou moins rapides, et plus ou moins efficaces selon le type de jeu de données utilisé (voir doc MLPClassifier).

3. la fonction d'activation : à choisir selon le type de problème et par l'expérimentation (voir doc MLPClassifier).

4. la quantité de données d'apprentissage : pour un problème simple comme celui de cet exemple, il n'est pas nécéssaire d'utiliser le jeu de données au complet, il peut être intéressant d'étudier à partir de quelle quantité de données le score n'évolue plus malgré l'ajout de données supplémentaires.

5. une visualisation graphique peut permettre de mieux comprendre l'impact des choix de ces paramètres (la librairie Matplotlib est un outil adapté), il faudra en premier lieu l'importer en haut de notre programme à l'aide de la ligne :

   ```
   import matplotlib.pyplot as plt
   ```

   Plusieurs types de visualisations sont envisageables :

   (a) en affichant les résultats sur le jeu de test sous forme de nuages de points colorés à l'aide de scatter plot.

   Exemple d'affichage des 100 premiers points et de leurs couleurs en fonction de la classe :

   ```
   plt.figure() # à mettre pour chaque figure
   plt.scatter(valeurs[0:100][:, 0], valeurs[0:100][:, 1], c=classe[0:100], cmap=plt.cm.bwr, edgecolors='black')
   plt.gca().set_aspect('equal', adjustable='box') # même ratio en X et en Y
   plt.show() # affichage (à ne mettre qu'une seule fois si plusieurs plot)
   ```

   Ressources utiles : doc matplotlib plot - liste des cartes de couleurs - doc scatter plot

   (b) en affichant les résultats du réseau entrainé sur tout l'espace du jeu de données à l'aide d'une carte de contours à l'aide de contourf :

   ```
   range_xy = np.arange(-0.1, 1.1, 0.05)  # bornes min/max en x et y du graphique

   image = [] # échantillonnage de la couleur de fond
   for x in range_xy:
       for y in range_xy:
           image.append([x, y])

   # utilisation du réseau entrainé sur chaque coordonnée de l'image de fond
   out = mlp.predict_proba(np.asarray(image))
   Z = out[:, 1]  # on récupère les valeurs pour construire l'image de fond
   Z = Z.reshape((len(range_xy), len(range_xy)))  # mise en forme des valeurs du fond

   plt.contourf(range_xy, range_xy, Z, cmap=plt.cm.bwr, alpha=.6)
   plt.scatter(valeurs[:, 0], valeurs[:, 1], c=classe, cmap=plt.cm.bwr, edgecolors='black')
   plt.show() # affichage
   ```

Partie 2 : Classification de matériaux.



La base de données que nous allons utiliser dans ce sujet a été construite par un centre de recherche en criminalistique afin d'identifier des débris de verre (par exemple sur une scène de crime).

6 types de verres sont présents dans la bases de données au travers de 214 entrées, représentés par 9 caractéristiques : l'indice de réfraction et le pourcentage de composition parmi 8 matériaux :

8 entrées : Indice de réfraction
Na : Sodium
Mg : Magnésium
Al : Aluminum
Si : Silicium
K : Potassium
Ca : Calcium
Ba : Béryllium
Fe : Fer

1 sortie :
0 : Verre de fenêtre de bâtiment (flotté)
1 : Verre de fenêtre de bâtiment (non-flotté)
2 : Verre de fenêtre de véhicule
3 : Verre de récipient de conservation
4 : Verre de vaisselle
5 : Verre de tête d'ampoule

Exemple de valeurs :

IR	Na	Mg	Al	Si	K	Ca	Ba	Fe	Type
1.52101	13.64	4.49	1.1	71.78	0.06	8.75	0	0	1
1.52369	13.44	0	1.58	72.22	0.32	12.24	0	0	5
...	...	...	...	...	...	...	...	...	...

Dans l'exemple du fichier `sujet2.py` on charge les données et on lance un apprentissage sur un seul neurone et sans jeu de test. À partir de cet exemple plusieurs points sont à explorer :

1. les hyperparamètres sont à choisir (voir partie 1) expliquez votre démarche.
2. dans une classification avec de nombreuses entrées et de nombreuses classes possibles, il est plus complexe de visualiser les résultats, pour cela une matrice de confusion est typiquement utilisée, elle permet de savoir dans les cas où une instance est mal classée avec quelle autre classe elle est confondue, cela permet d'analyser d'où viennent les erreurs.
   Voici un exemple de code permettant d'afficher cette matrice, implémenter cet affichage dans votre code :

```
# imports nécessaires
from sklearn.metrics import confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# [...] ici le reste du code

pred = mlp.predict(valeurs) # on calcule des classes sur toutes les entrées de la liste valeurs
cm = confusion_matrix(classe, pred) # construction de la matrice de confusion

# affichage de la matrice de confusion
cm_display = ConfusionMatrixDisplay(cm)
cm_display.plot(cmap=plt.cm.Blues)
plt.show()
```

3. Dans cet exemple il serait difficile de voir si il n'y a pas sur-apprentissage, pour éviter cela, il faut rajouter un jeu de test, la librairie sklearn donne une fonction (train_test_split) qui permet de faire une séparation aléatoire d'un jeu de données afin de construire un jeu de test, voici un exemple d'utilisation :

```
valeurs_train, valeurs_test, classe_train, classe_test = train_test_split(valeurs, classe, test_size=0.33)
```

   Que remarquez vous sur les scores après la séparation du jeu de données ? Réajustez vos hyper-paramètre pour avoir un bon score sur le jeu de test.

4. Effectuez plusieurs apprentissages (dans une boucle python) avec un nouveau tirage du jeu d'apprentissage à chaque fois afin de construire des scores moyens sur de nombreux tests, quelle classe est la plus dure à reconnaître en moyenne ?

Partie 3 : Consommation d'énergie et données météo.

Le campus des Arts et Métiers d'Aix-en-Provence est équipé de systèmes de relevés de consommation d'énergie, cela permet de suivre heure par heure l'énergie électrique consommée sur la totalité du campus. Cette consommation d'énergie dépend de beaucoup de paramètres, de l'occupation des lieux (jours/nuits/week-end/vacances), mais également des différentes activités en cours, mais aussi de la météo, et de la période de l'année, etc. Plusieurs données météorologiques de la ville d'Aix ont été réunies afin construire une base (venant de Météo-France), également heure par heure, contenant la température, la vitesse du vent, l'orientation du vent, et l'humidité.

Toutes ces données ont été réunies en une seule base de données, sur la durée d'une année, heure par heure : du 13 octobre 2021 au 12 octobre 2022, soit 8725 entrées (il manque cependant 17 entrées à différents moments pour cause probable de capteurs défectueux).

Voici la forme de la base de données :

Jour	Mois	Année	Heure	Vitesse vent	Orientation vent	Température	Humidité	Conso
13.0	10.0	2021.0	0.0	0.0	0.0	8.3	86.0	50.0
13.0	10.0	2021.0	1.0	0.5	360.0	7.8	87.5	50.0
13.0	10.0	2021.0	2.0	0.0	0.0	7.4	89.0	51.0
...	...	...	...	...	...	...	...	...

Le fichier de départ `sujet3.py` ouvre la base, et donne un exemple simple d'utilisation d'un réseau de régression (MLPRegressor).

1. Pour mieux appréhender la base de données nous pouvons construire des graphiques des différentes valeurs. Par exemple sous forme de courbes :

```
plt.plot(conso)
plt.show() # affichage du graphique
```

   Afin de mieux voir la temporalité des données, nous pouvons utiliser les mois sur l'axe des abscisses :

```
mois = np.asarray(["jan", "fév", "mars", "avr", "mai", "juin", "juil", "août", "sept", "oct", "nov", "déc"])
# on récupère les index des débuts de mois
index_debut_de_mois = np.where((valeurs[:, 0] == 1) & (valeurs[:, 3] == 0))[0]
ax = plt.gca()
ax.set_xticks(index_debut_de_mois) # un tick pour chaque début de mois
# on utilise la str du tableau mois aux indices de index_debut_de_mois
ax.set_xticklabels(mois[valeurs[:, 1][index_debut_de_mois].astype(int)-1], rotation=45)

ax.plot(conso) # on affiche la conso
plt.show() # affichage du graphique
```

   On peut visualiser toutes les données en remplaçant `ax.plot(conso)` par `ax.plot([valeurs[:, i])` avec `i` la ième données à afficher.

2. À partir du fichier `sujet3.py`, nous pouvons explorer plusieurs possibilités, l'idée la plus simple est de tenter de de prédire la consommation actuelle à partir des données météo actuelles. Développez un modèle permettant de faire cette prédiction et choisissez les hyper-paramètres.

Afin de réduire les instabilités de convergence du réseau, nous pouvons normaliser les entrées afin de les borner entre 0 et 1 :

```
for i in range(8):
    valeurs[:, i] = valeurs[:, i] / np.max(valeurs[:, i])
```

3. Les paramètres d'entrée peuvent avoir un impact plus moins important, nous pouvons étudier cet impact en retirant des entrées de l'apprentissage afin de voir l'influence de la présence ou l'absence de certaines entrées.

   Il existe plusieurs façons de supprimer une entrée : il est possible de l'enlever du tableau d'entrées, mais il est aussi possible de la rendre inutilisable en effectuant des permutations aléatoires dessus, voici un exemple de code permettant d'effectuer une permutation aléatoire sur une entrée d'un tableau de valeurs :

   ```
   # on fait une copie de toutes les entrées
   valeurs_2 = np.copy(valeurs)
   # on effectue une permutation aléatoire sur la colonne d'indice 0
   valeurs_2[:, 0] = np.random.permutation(valeurs_2[:, 0])
   ```

   Quelles sont les entrées qui ont le plus d'impact dans la prédiction ?

4. Certaines entrées peuvent se compenser (donner la même information au réseau), afin de savoir lesquelles tentez de retirer plusieurs entrées à la fois, quelles combinaisons d'entrées sont les plus impactantes ? Avec quelles entrées au minimum il est possible de faire une prédiction ? Pourquoi ?